SIGECOIM

Sistema de Información para la Gestión Administrativa del Comodato de Instrumentos Musicales

Documento técnico de estructura del proyecto: organización de carpetas, arquitectura backend/frontend y explicación línea por línea de los fragmentos de código más representativos de cada capa. Generado como referencia de onboarding para desarrolladores.

Monorepo	API (Node/Express/Sequelize) + Cliente (React/Vite), mitades independientes
Backend	Node.js 18+ · Express · Sequelize ORM · MySQL · JWT · bcryptjs
Frontend	React 19 · Vite · MUI · librería de diseño interna anteriority-ui
Dominio	Núcleo de la Sinfónica Infantil — inventario de instrumentos y contratos de comodato

1. Descripción General

SIGECOIM digitaliza el inventario de instrumentos musicales y los contratos de comodato (préstamo gratuito) que la institución otorga a estudiantes de la Sinfónica Infantil. Sustituye hojas de cálculo y registros en papel por un panel administrativo que garantiza trazabilidad: qué instrumento está prestado, a qué estudiante, bajo la responsabilidad de qué representante legal, y desde cuándo.

Conceptos clave del dominio

- **Comodato:** contrato legal de préstamo gratuito de un instrumento a un estudiante.
- **Correlativo de contrato (folio_contrato):** código único por año, ej. `COM-2026-001`, generado secuencialmente.
- **Código de inventario:** identificador físico institucional grabado en cada instrumento.
- **Comodatario:** representante legal que firma el comodato y asume la responsabilidad civil cuando el estudiante es menor de edad.
- **Solicitud:** mecanismo de autoservicio donde el propio estudiante solicita un préstamo, sujeto a aprobación de un analista/coordinador.

Roles y permisos

Acción	Analista de Bienes	Coordinador
Login / módulos operativos	Sí	Sí
CRUD Estudiantes / Instrumentos / Comodatos	Sí	Sí
Procesar devoluciones	Sí	Sí
Administración de Usuarios (CRUD)	Sí	No

En frontend esto se traduce en `puedeAdministrarUsuarios` (derivado en `AuthContext`) y en backend en el middleware `requireRol('ANALISTA_BIENES', 'COORDINADOR')`.

2. Arquitectura General

El repositorio es un **monorepo con dos mitades independientes**: una API en la raíz (Node/Express/Sequelize) y un cliente SPA en `client/` (React/Vite). No comparten `node_modules` ni proceso — se instalan y arrancan por separado.

2.1 Backend — capas

Express en capas clásicas: `routes/` → `controllers/` → `models/` (Sequelize), con un middleware de autenticación JWT transversal.

- **Esquema sin migraciones:** `config/db.js` llama `sequelize.sync({ alter: true })` en cada arranque — el esquema se auto-corrige desde las definiciones de los modelos. No hay archivos de migración que escribir.
- `src/models/sync.js` es un script independiente (no se importa desde `server.js`) que se corre una sola vez por entorno nuevo para crear tablas y sembrar datos de prueba.
- `src/models/index.js` es la **única fuente de verdad de las asociaciones** (Representante → Estudiante → Comodato → Instrumento/Devolución/Usuario/Solicitud) y el único lugar desde donde los controladores deben importar modelos.
- **Dos formas de JWT distintas:** `requireTipo('STAFF'|'ESTUDIANTE')` filtra por tipo de cuenta (login de personal vs. estudiantes), mientras que `requireRol('ANALISTA_BIENES', 'COORDINADOR')` filtra por rol dentro del personal. No son intercambiables.

2.2 Frontend — capas

- React 19 + Vite, enrutado con React Router, UI con MUI.
- **Los componentes de interfaz vienen del paquete** `anteriority-ui` (librería de diseño interna compartida), no de primitivos locales — se prioriza sobre MUI manual cuando hay un equivalente.
- `anteriority-ui` también provee infraestructura, no solo visual: `client/src/services/apiClient.js` configura el cliente HTTP compartido vía `configureApiClient` en lugar de instanciar un axios propio.
- **Resolución dinámica de URL base de la API:** localhost/127.0.0.1 → puerto 3003 local; host `*.devtunnels.ms` → mismo túnel cambiando el sufijo `-5173` por `-3003`; cualquier otro host (IP de LAN) → mismo host en el puerto 3003. Esto permite probar desde un celular en la misma Wi-Fi o por dev tunnel sin tocar código.
- Token y usuario se persisten en `localStorage` (`sigecoim_token` / `sigecoim_usuario`) y se leen desde `AuthContext`, que expone `usuario`, `login`, `logout` y el flag derivado `puedeAdministrarUsuarios`.
- Guardas de ruta: `ProtectedRoute` exige cualquier usuario logueado; `AdminRoute` además exige `puedeAdministrarUsuarios`, redirigiendo a `/estudiantes` si no se cumple.

2.3 Regla de negocio transversal

Un `Instrumento` en estado `PRESTADO` no puede adjuntarse a un nuevo `Comodato`. Eliminar un comodato activo debe liberar el instrumento a `DISPONIBLE`. Aprobar una `Solicitud` debe crear el `Comodato` y cambiar el estado del instrumento dentro de la misma transacción de base de datos. El patrón de referencia está en `comodatoController.createComodato` / `generarFolioContrato` (ver sección 4).

3. Estructura de Carpetas

3.1 Backend — raíz del repositorio

```
SIGECOIM/
|-- server.js                          Arranque: Express, CORS, monta las rutas, connectDB()
|-- package.json
|-- .env                               PORT, DB_HOST, DB_USER, DB_PASS, DB_NAME, JWT_SECRET
`-- src/
    |-- config/
    | `-- db.js                        Conexion Sequelize + sync({ alter: true })
    |-- middlewares/
    | `-- authMiddleware.js            verifyToken, requireTipo, requireRol
    |-- models/
    | |-- index.js                     Unica fuente de verdad de asociaciones (export central)
    | |-- sync.js                      Script manual: crear tablas + seed (1 vez por entorno)
    | |-- Usuario.js                   Personal (STAFF): credenciales y rol
    | |-- Representante.js             Comodatario (representante legal)
    | |-- Estudiante.js                Beneficiario del prestamo
    | |-- Instrumento.js               Inventario fisico + disponibilidad/conservacion
    | |-- Comodato.js                  Contrato de prestamo (activo/vencido/finalizado)
    | |-- Devolucion.js                Registro de retorno fisico de un instrumento
    | |-- Solicitud.js                 Solicitud de prestamo por autoservicio del estudiante
    | |-- HistorialUbicacion.js        Trazabilidad de movimientos de un instrumento
    | `-- Configuracion.js             Parametros editables (ej. dias de alerta)
    |-- controllers/
    | |-- authController.js            login
    | |-- comodatoController.js        CRUD + folio + devolucion, todo transaccional
    | |-- configuracionController.js
    | |-- estudianteController.js
    | |-- instrumentoController.js
    | |-- solicitudController.js       aprobarSolicitud -> crea Comodato + mueve instrumento
    | `-- usuarioController.js
    `-- routes/                        authRoutes, comodatoRoutes, configuracionRoutes,
        (7 archivos)                   estudianteRoutes, instrumentoRoutes, solicitudRoutes, usuarioRoutes
```

Todas las rutas anteriores están montadas en `server.js` *bajo* `/api/<recurso>` *(incluyendo* `/api/solicitudes`*).*

3.2 Frontend — client/src/

```
client/
|-- index.html
|-- package.json                       scripts: dev, build, lint (oxlint), preview
`-- src/
    |-- main.jsx                       Entry point de Vite/React
    |-- App.jsx                        Enrutamiento raiz (React Router)
```

```
    |-- assets/                          Logos e imagenes estaticas (ELSISTEMA.png,
                                         logo1.png)
    |-- components/
    | `-- FormMessage.jsx                Mensaje de error/validacion reutilizable
    |-- constants/
    | `-- text.js                        Textos/labels centralizados de la UI
    |-- context/
    | `-- AuthContext.jsx                usuario, login, logout, permisos
    |-- layout/
    | |-- AppLayout.jsx                  Shell con sidebar/topbar de paginas protegidas
    | `-- constants.js
    |-- routes/
    | `-- ProtectedRoute.jsx             ProtectedRoute / AdminRoute (guardas de sesion)
    |-- services/                        Un archivo por recurso del backend, sobre
                                         apiClient.js
    | |-- apiClient.js                   configureApiClient + resolucion de baseURL
    | `-- (5 archivos)                   auth, comodatos, configuracion, estudiantes,
                                         usuarios
    |-- theme/
    | |-- theme.js                       Tema MUI (colores corporativos, tipografias)
    | `-- tokens.js                      Design tokens base
    |-- utils/
    | |-- edad.js                        Calculo de edad / mayoria de edad del estudiante
    | `-- pdf/                           Generacion de PDFs (contrato y reporte de comodatos)
    `-- pages/modules/
        |-- login/, dashboard/,
estudiantes/,
        |-- instrumentos/, comodatos/,
        |-- usuarios/                    Protegido por AdminRoute
        `-- configuracion/
```

Cada carpeta bajo `pages/modules/` *compone una página completa (listado + formulario + tabla) a partir de componentes de* `anteriority-ui` *y de los servicios correspondientes.*

4. Código Backend — explicado línea por línea

Esta sección toma fragmentos reales de los archivos del backend y explica, línea por línea, qué hace cada una. Está pensada para alguien que nunca vio el código y necesita entenderlo sin ejecutar nada.

4.1 server.js — archivo de arranque (raíz del repo)

`import express from 'express';`	Importa el framework Express, usado para crear el servidor HTTP.
`import cors from 'cors';`	Importa el middleware que controla qué dominios pueden llamar a la API.
`import { connectDB } from './src/config/db.js';`	Trae la función que abre la conexión a MySQL vía Sequelize.
`import authRoutes from './src/routes/authRoutes.js';`	Trae el router de login (y los otros 6 routers, uno por recurso).
`const app = express();`	Crea la aplicación Express; a partir de aquí se configura todo.
`app.use(cors({ origin: fn, ... }));`	Activa CORS: la función 'origin' decide qué peticiones aceptar según el dominio que llama.
`  if (!origin) return callback(null, true);`	Permite peticiones sin origen (Postman, apps móviles, curl).
`  const isLocal = origin.includes('localhost') || ...`	Marca como válido el origen si es localhost, 127.0.0.1 o una IP de red local (192.168.x.x / 10.x.x.x).
`  const isDevTunnel = origin.includes('devtunnels.ms');`	Marca como válido cualquier túnel de desarrollo de VS Code.
`  if (isLocal || isDevTunnel) callback(null, true);`	Si pasa alguna de las dos condiciones, deja pasar la petición.
`  else callback(new Error(...), false);`	Si no, Express responde con error de CORS y bloquea la petición.
`app.use(express.json());`	Permite que Express lea automáticamente el body JSON de las peticiones.
`connectDB();`	Abre la conexión a la base de datos y ejecuta sequelize.sync({ alter: true }) dentro.
`app.use('/api/auth', authRoutes);`	Monta el router de autenticación bajo el prefijo /api/auth.
`app.use('/api/comodatos', comodatoRoutes);`	Monta el router de comodatos bajo /api/comodatos (y así con cada recurso).
`app.get('/api/health', (req, res) => {...});`	Endpoint simple para verificar que el servidor y la DB responden.
`app.listen(PORT, () => {...});`	Inicia el servidor HTTP en el puerto configurado (por defecto 3003).

4.2 src/middlewares/authMiddleware.js — verificación de JWT

`const secret = process.env.JWT_SECRET || '...';`	Define la clave secreta para firmar/verificar el JWT, con un valor por defecto si falta en .env.
`export const verifyToken = (req, res, next) => {`	Middleware que corre antes del controlador en toda ruta protegida.
`  const header = req.headers.authorization;`	Lee el encabezado Authorization enviado por el frontend.
`  if (!header || !header.startsWith('Bearer ')) ...`	Si no viene el encabezado o no tiene el formato 'Bearer <token>', responde 401.
`  const token = header.split(' ')[1];`	Extrae solo el token, descartando la palabra 'Bearer'.
`  req.user = jwt.verify(token, secret);`	Verifica la firma del token y, si es válido, decodifica su contenido en req.user.

`  next();`	Token válido: continúa hacia el siguiente middleware o el controlador.
`  catch { return res.status(401)...}`	Si el token es inválido o expiró, corta la petición con 401.
`export const requireTipo = (...tipos) => (req,res,next) => {`	Fábrica de middleware: recibe los tipos de cuenta permitidos ('STAFF', 'ESTUDIANTE').
`  if (!req.user || !tipos.includes(req.user.tipo)) ...`	Si el tipo de cuenta del token no está en la lista permitida, responde 403 (prohibido).
`  next();`	Si el tipo coincide, deja pasar la petición.
`export const requireRol = (...roles) => (req,res,next) => {`	Igual que el anterior, pero filtra por rol de personal ('ANALISTA_BIENES', 'COORDINADOR').
`  if (!req.user || !roles.includes(req.user.rol)) ...`	Si el rol del usuario de staff no está permitido para esta ruta, responde 403.

4.3 src/controllers/comodatoController.js — registrar un préstamo (createComodato)

Este es el patrón transaccional que hay que preservar siempre que se toquen comodatos/instrumentos: todo ocurre dentro de una única transacción de base de datos, y si algo falla se revierte todo con rollback.

`const t = await sequelize.transaction();`	Abre una transacción de base de datos: todo lo que siga puede deshacerse de una sola vez.
`const { folio_contrato, estudiante_id, instrumento_id, ... } = req.body;`	Extrae del body los datos enviados por el formulario del frontend.
`if (!folio_contrato || !fecha_devolucion_prevista || ...) {...}`	Valida que los campos obligatorios estén presentes; si falta alguno, responde 400 y hace rollback.
`const existeFolio = await Comodato.findOne({ where: { folio_contrato } });`	Busca si ya existe un comodato con ese mismo correlativo, para evitar duplicados.
`const estudiante = await Estudiante.findByPk(estudiante_id);`	Verifica que el estudiante seleccionado realmente exista en la base de datos.
`const prestamosActivos = await Comodato.count({ where: { estudiante_id, estado: 'ACTIVO' }, transaction: t });`	Cuenta cuántos préstamos ACTIVOS tiene ya ese estudiante, dentro de la misma transacción.
`if (prestamosActivos >= MAX_INSTRUMENTOS_POR_ESTUDIANTE) {...}`	Si ya alcanzó el límite permitido de instrumentos simultáneos, rechaza la operación.
`const instrumento = await Instrumento.findByPk(instrumento_id, { transaction: t });`	Carga el instrumento solicitado dentro de la transacción (lo 'bloquea' lógicamente para esta operación).
`if (instrumento.estado_disponibilidad !== 'DISPONIBLE') {...}`	Regla de negocio clave: un instrumento que no está DISPONIBLE no puede prestarse de nuevo.
`if (instrumento.estado_conservacion === 'MALO') {...}`	Tampoco se puede prestar un instrumento marcado en mal estado de conservación.
`const nuevoComodato = await Comodato.create({ ... }, { transaction: t });`	Crea el registro del comodato con estado inicial 'ACTIVO', dentro de la transacción.
`await instrumento.update({ estado_disponibilidad: 'PRESTADO' }, { transaction: t });`	Cambia el estado del instrumento a PRESTADO — esto es lo que impide prestarlo dos veces.
`await t.commit();`	Confirma la transacción: ahora sí se guardan permanentemente el comodato nuevo y el cambio de estado.
`res.status(201).json(nuevoComodato);`	Responde al frontend con el comodato recién creado.
`catch (error) { await t.rollback(); ... }`	Si algo falla en cualquier paso anterior, deshace todos los cambios hechos en la transacción.

Nota: `deleteComodato` *aplica la regla inversa — si el comodato estaba ACTIVO, libera el instrumento a DISPONIBLE antes de destruir el registro.* `generarFolioContrato(t)` *cuenta los comodatos del año actual dentro de la misma transacción y arma el correlativo* `COM-{año}-{secuencial}`.

4.4 src/models/index.js — asociaciones entre modelos

`Representante.hasMany(Estudiante, { foreignKey: 'representante_id', as: 'estudiantes' });`	Un representante puede tener muchos estudiantes a su cargo.

`Estudiante.belongsTo(Representante, { foreignKey: 'representante_id', as: 'representante' });`	Cada estudiante pertenece (como máximo) a un representante legal.
`Estudiante.hasMany(Comodato, { foreignKey: 'estudiante_id', as: 'comodatos' });`	Un estudiante puede tener varios comodatos a lo largo del tiempo (históricos y activos).
`Instrumento.hasMany(Comodato, { foreignKey: 'instrumento_id', as: 'comodatos' });`	Un instrumento puede aparecer en varios comodatos a lo largo de su vida útil.
`Comodato.hasOne(Devolucion, { foreignKey: 'comodato_id', as: 'devolucion' });`	Cada comodato tiene a lo sumo una devolución asociada, cuando se cierra el préstamo.
`Comodato.hasOne(Solicitud, { foreignKey: 'comodato_id', as: 'solicitud' });`	Si el comodato nació de una solicitud aprobada, queda vinculado a esa solicitud.
`export { sequelize, Usuario, Representante, Estudiante, ... };`	Punto único de exportación: los controladores siempre importan los modelos desde aquí.

5. Código Frontend — explicado línea por línea

5.1 client/src/services/apiClient.js — URL de la API según el entorno

`import { configureApiClient } from 'anteriority-ui/common/api/client';`	Importa el configurador del cliente HTTP compartido (no se usa axios propio).
`function resolveBaseURL() {`	Función que decide a qué URL de backend apuntar según desde dónde se abre la app.
`  const { hostname, protocol } = window.location;`	Lee el dominio y protocolo actuales del navegador (ej. 'localhost', 'http:').
`  if (hostname === 'localhost' || hostname === '127.0.0.1') {`	Caso 1: se abre en la propia PC de desarrollo.
`    return 'http://localhost:3003/api';`	Apunta directo al backend local en el puerto 3003.
`  if (hostname.endsWith('.devtunnels.ms')) {`	Caso 2: se abre a través de un dev tunnel de VS Code.
`    return `${protocol}//${hostname.replace('-5173','-3003')}/api`;`	Usa el mismo túnel pero cambia el sufijo de puerto del frontend (-5173) por el del backend (-3003).
`  return `${protocol}//${hostname}:3003/api`;`	Caso 3: cualquier otro host (ej. IP de red local) — mismo host, puerto 3003.
`export function setupApiClient() {`	Función que se llama una vez al arrancar la app para configurar el cliente HTTP.
`  configureApiClient({ baseURL: resolveBaseURL(), getAuthToken: () => ... });`	Configura la URL base calculada arriba y cómo obtener el token JWT guardado.

5.2 client/src/context/AuthContext.jsx — sesión global

`const [usuario, setUsuario] = useState(readUsuario);`	Estado de React con el usuario logueado, inicializado leyendo localStorage.
`const login = async (email, password) => {`	Función de login por email/password.
`  const { token, usuario: nuevoUsuario } = await loginRequest(email, password);`	Llama al backend (/api/auth/login) y recibe el JWT más los datos del usuario.
`  localStorage.setItem(TOKEN_KEY, token);`	Guarda el token en localStorage bajo la clave 'sigecoim_token'.
`  localStorage.setItem(USER_KEY, JSON.stringify(nuevoUsuario));`	Guarda también los datos del usuario, serializados como JSON.
`  setUsuario(nuevoUsuario);`	Actualiza el estado de React para que toda la app reaccione al nuevo usuario logueado.
`const logout = () => { localStorage.removeItem(...); setUsuario(null); };`	Borra el token y el usuario de localStorage y limpia el estado en memoria.
`const puedeAdministrarUsuarios = usuario?.rol === 'ADMINISTRADOR' || usuario?.rol === 'ANALISTA_BIENES';`	Calcula si el usuario actual puede entrar al módulo de administración de usuarios.
`return <AuthContext.Provider value={{ usuario, login, ... }}>{children}</AuthContext.Provider>;`	Expone todo lo anterior a cualquier componente hijo mediante el contexto de React.

5.3 client/src/routes/ProtectedRoute.jsx — guardas de ruta

`const { usuario } = useAuth();`	Lee el usuario actual desde AuthContext.
`if (!usuario) return <Navigate to="/login" replace />;`	Si no hay sesión activa, redirige al login en vez de mostrar la página pedida.
`return children;`	Si hay sesión, renderiza la página protegida normalmente.
`const { puedeAdministrarUsuarios } = useAuth();`	(En AdminRoute) además del usuario, lee el flag de permiso de administración.
`if (!puedeAdministrarUsuarios) return <Navigate to="/estudiantes" replace />;`	Si el usuario no tiene el rol adecuado, lo redirige a /estudiantes en vez de dejarlo entrar.

`ProtectedRoute` *envuelve cualquier página que requiera sesión;* `AdminRoute` *envuelve específicamente el módulo de Usuarios, que solo puede ver el Analista de Bienes (o un Administrador).*

5.4 Convención de servicios por recurso

Cada archivo en `client/src/services/` corresponde 1:1 a un recurso del backend (`comodatos.js` habla con `/api/comodatos`, etc.) y expone funciones simples que llaman al cliente HTTP configurado en `apiClient.js`. Las páginas en `pages/modules/<recurso>/` consumen ese servicio y componen listado, formulario y tabla usando primitivas de `anteriority-ui`.

6. Comandos de Desarrollo

Backend (desde la raíz del repositorio)

```
pnpm install
node src/models/sync.js # una sola vez: crea tablas + siembra datos (requiere MySQL/XAMPP
activo)
pnpm dev # node --watch server.js -> http://localhost:3003
pnpm start # node server.js (sin watch)
```

Frontend (desde client/)

```
pnpm install
pnpm dev # vite -> http://localhost:5173
pnpm build
pnpm lint # oxlint
pnpm preview
```

Variables de entorno (.env en la raíz)

- `PORT, DB_HOST, DB_USER, DB_PASS, DB_NAME, JWT_SECRET` — requeridas siempre.

No existe suite de pruebas automatizadas en ninguno de los dos paquetes al momento de este documento.